

Analytical Problem Solving

Given Problem

A delivery robot is working in a 5×5 warehouse grid.

- Starting cell: **(1,1)**
- Goal cell: **(5,5)**
- Blocked cells: **(2,2), (2,3), (3,3), (4,2), (4,4)**
- Allowed movements: **Up, Down, Left, Right**
- Cost of each movement: **1**

For BFS and DFS, I use the movement order:

Up $\rightarrow$ Down $\rightarrow$ Left $\rightarrow$ Right

This makes the node expansion order clear and consistent.

(a) PEAS Description

PEAS stands for Performance Measure, Environment, Actuators and Sensors.

1. Performance Measure

The main aim of the robot is to deliver the package to **(5,5)**.

The robot should:

- Reach the destination successfully.
- Avoid blocked cells.
- Stay inside the warehouse.
- Use the minimum possible number of movements.
- Complete the task efficiently.

Since every movement costs 1, a shorter path has a lower total cost.

2. Environment

The environment is a 5×5 warehouse grid.

It contains:

- Starting position (1,1)
- Destination (5,5)
- Blocked cells caused by shelves:
 - (2,2) ◦
 - (2,3) ◦
 - (3,3) ◦
 - (4,2) ◦
 - (4,4)
- Free cells where the robot can move.

The robot can move only horizontally or vertically.

3. Actuators

The robot's actuators allow it to:

- Move Up
- Move Down
- Move Left
- Move Right

4. Sensors

The robot needs sensors to know:

- Its current position.
- Whether nearby cells are blocked.

- Whether it is at the boundary.
- Whether it has reached the destination.

PEAS	Description
Performance Measure	Reach (5,5) safely with minimum movement cost
Environment	5 × 5 warehouse grid with blocked cells
Actuators	Up, Down, Left, Right
Sensors	Position, obstacle, boundary and goal detection

Conclusion: This PEAS model describes what the robot has to achieve, where it works, how it moves and what information it receives.

(b) Type of Intelligent Agent

Goal-Based Agent

The most suitable agent for this problem is a Goal-Based Agent.

The robot has a specific goal: reach cell (5,5) from (1,1).

It has to consider different possible movements and choose a sequence that leads to the destination without entering blocked cells.

- A Simple Reflex Agent mainly reacts to the current situation and does not properly plan a complete route.
- A Model-Based Agent can maintain information about the environment, but the main requirement here is reaching a particular goal.
- A Goal-Based Agent chooses actions based on whether they help achieve the destination.
- A Utility-Based Agent would be more suitable if factors such as fuel, time, risk or energy had to be compared.

Therefore, the Goal-Based Agent is the best choice because the robot has a clearly defined destination and must plan a path to reach it.

(c) Search Problem Formulation

The warehouse problem can be represented as a search problem using five components.

1. Initial State

The robot starts at (1,1).

2. Goal State

The robot must reach (5,5).

3. Actions

The possible actions are:

Up, Down, Left, Right

An action is allowed only if the resulting cell:

- Is inside the 5×5 grid.
- Is not blocked.

4. Transition Model

The transition model tells us the new position after each movement.

For a position (r,c):

$Up=(r-1,c)$

$Down=(r+1,c)$

$Left=(r,c-1)$

$Right=(r,c+1)$

For example: 5. Path Cost $(1, 1) \text{Down} \rightarrow (2, 1)$

Every movement costs 1.

Therefore:

$$\text{Path Cost} = \text{Number of movements} \times 1$$

So, the objective is to find a path from (1,1) to (5,5) with minimum cost.

(d) Breadth-First Search (BFS)

Program:

```
from collections import deque
n = 5
start = (1, 1)
goal = (5, 5)

blocked = {(2, 2), (2, 3), (3, 3), (4, 2), (4, 4)}

moves = [
    (-1, 0), # Up
    (1, 0), # Down
    (0, -1), # Left
    (0, 1) # Right
]
queue = deque([(start, [start])])
visited = {start}
step = 1
while queue:
    current, path = queue.popleft()
    if current == goal:
        print("\nGoal Reached!")
        print("Final Path:", path)
        print("Total Cost:", len(path) - 1)
        break
    for dr, dc in moves:
        r = current[0] + dr
        c = current[1] + dc
        next_cell = (r, c)
        if (1 <= r <= n and 1 <= c <= n
            and next_cell not in blocked
            and next_cell not in visited):
            visited.add(next_cell)
            queue.append((next_cell, path + [next_cell]))

    print("Step", step, "Expanded:", current)
    print("Queue:", [node for node, path in queue])
    step += 1
```

Output:

```
Step 1 Expanded: (1, 1)
Queue: [(2, 1), (1, 2)]
Step 2 Expanded: (2, 1)
Queue: [(1, 2), (3, 1)]
Step 3 Expanded: (1, 2)
Queue: [(3, 1), (1, 3)]
Step 4 Expanded: (3, 1)
Queue: [(1, 3), (4, 1), (3, 2)]
Step 5 Expanded: (1, 3)
Queue: [(4, 1), (3, 2), (1, 4)]
Step 6 Expanded: (4, 1)
Queue: [(3, 2), (1, 4), (5, 1)]
Step 7 Expanded: (3, 2)
Queue: [(1, 4), (5, 1)]
Step 8 Expanded: (1, 4)
Queue: [(5, 1), (2, 4), (1, 5)]
Step 9 Expanded: (5, 1)
Queue: [(2, 4), (1, 5), (5, 2)]
Step 10 Expanded: (2, 4)
Queue: [(1, 5), (5, 2), (3, 4), (2, 5)]
Step 11 Expanded: (1, 5)
Queue: [(5, 2), (3, 4), (2, 5)]
Step 12 Expanded: (5, 2)
Queue: [(3, 4), (2, 5), (5, 3)]
Step 13 Expanded: (3, 4)
Queue: [(2, 5), (5, 3), (3, 5)]
Step 14 Expanded: (2, 5)
Queue: [(5, 3), (3, 5)]
Step 15 Expanded: (5, 3)
Queue: [(3, 5), (4, 3), (5, 4)]
Step 16 Expanded: (3, 5)
Queue: [(4, 3), (5, 4), (4, 5)]
Step 17 Expanded: (4, 3)
Queue: [(5, 4), (4, 5)]
Step 18 Expanded: (5, 4)
Queue: [(4, 5), (5, 5)]
Step 19 Expanded: (4, 5)
Queue: [(5, 5)]

Goal Reached!
Final Path: [(1, 1), (2, 1), (3, 1), (4, 1), (5, 1), (5, 2), (5, 3), (5, 4), (5, 5)]
Total Cost: 8
```

(e) Depth-First Search (DFS) Program:

```
n = 5
start = (1, 1)
goal = (5, 5)
blocked = {(2, 2), (2, 3), (3, 3), (4, 2), (4, 4)}
moves = [
    (-1, 0), # Up
    (1, 0), # Down
    (0, -1), # Left
    (0, 1) # Right
]
stack = [(start, [start])]
visited = {start}
step = 1

while stack:
    current, path = stack.pop()

    if current == goal:
        print("\nGoal Reached!")
        print("Final Path:", path)
        print("Total Cost:", len(path) - 1)
        break

    # Reverse so that Up, Down, Left, Right
    # are explored in the required order
    for dr, dc in reversed(moves):
        r = current[0] + dr
        c = current[1] + dc
        next_cell = (r, c)

        if (1 <= r <= n and 1 <= c <= n
            and next_cell not in blocked
            and next_cell not in visited):

            visited.add(next_cell)
            stack.append((next_cell, path + [next_cell]))

    print("Step", step, "Expanded:", current)
    print("Stack:", [node for node, path in reversed(stack)])
    step += 1
```

Output:

```
Step 1 Expanded: (1, 1)
Stack: [(2, 1), (1, 2)]
Step 2 Expanded: (2, 1)
Stack: [(3, 1), (1, 2)]
Step 3 Expanded: (3, 1)
Stack: [(4, 1), (3, 2), (1, 2)]
Step 4 Expanded: (4, 1)
Stack: [(5, 1), (3, 2), (1, 2)]
Step 5 Expanded: (5, 1)
Stack: [(5, 2), (3, 2), (1, 2)]
Step 6 Expanded: (5, 2)
Stack: [(5, 3), (3, 2), (1, 2)]
Step 7 Expanded: (5, 3)
Stack: [(4, 3), (5, 4), (3, 2), (1, 2)]
Step 8 Expanded: (4, 3)
Stack: [(5, 4), (3, 2), (1, 2)]
Step 9 Expanded: (5, 4)
Stack: [(5, 5), (3, 2), (1, 2)]

Goal Reached!
Final Path: [(1, 1), (2, 1), (3, 1), (4, 1), (5, 1), (5, 2), (5, 3), (5, 4), (5, 5)]
Total Cost: 8
```

(f) BFS vs DFS – 10 Marks

Feature	BFS	DFS
Data structure	Queue	Stack
Search method	Level by level	Deepest branch first
Complete	Yes	Yes for this finite grid with visited-state checking
Optimal	Yes, for uniform step costs	No, in general
Time complexity	$\mathcal{O}(b^d)$	$\mathcal{O}(b^m)$
Space complexity	$\mathcal{O}(b^d)$	$\mathcal{O}(bm)$
Path found here	Cost 8	Cost 8
Main advantage	Guarantees shortest path	Requires less memory
Main disadvantage	Uses more memory	May find a non-optimal path

Where:

- b = branching factor
- d = depth of the shallowest solution
- m = maximum search depth

For this problem, BFS is more suitable.

The reason is that every movement has the same cost of 1. BFS searches level by level, so it finds the shortest path first.

DFS also finds a path with cost 8 in this particular case, but this happens because of the chosen movement order. DFS cannot guarantee that its first solution will always be the shortest.

Conclusion:

The warehouse robot is best modeled as a Goal-Based Intelligent Agent. The problem can be represented as a search problem with (1,1) as the initial state and (5,5) as the goal state.

Both algorithms successfully reach the destination:

$$BFS\ Cost = 8$$

However, BFS is the better choice because *DFS* all *Cost* movements = 8 have equal cost and BFS guarantees the shortest path.

Therefore, the optimal delivery route is:

with: $(1, 1) \rightarrow (2, 1) \rightarrow (3, 1) \rightarrow (4, 1) \rightarrow (5, 1) \rightarrow (5, 2) \rightarrow (5, 3) \rightarrow (5, 4) \rightarrow (5, 5)$

8 moves and total cost 8